DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado de sistemas secuenciales con varios procesos**	**CÓDIGO:** **ATGP4**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Aprendizaje colaborativo. Tutorial.	TIPO:	Presencial	DURACIÓN:	100 minutos

CALENDARIO:	Sesión presencial S8.	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Modelado y test VHDL.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

PARTE I. Introducción

En esta actividad se realiza un tutorial en el que se trata el problema del modelado de sistemas secuenciales síncronos complejos con salidas que son función del estado de memoria o de las entradas síncronas del sistema. Se mostrará cómo este tipo de sistemas puede modelarse con dos procesos y se estudiará el modo en que un simulador VHDL ejecuta este tipo de modelos.

Todas estas cuestiones se ilustrarán usando como ejemplo la realización del modelo de un divisor de frecuencia.

La actividad se realizará por parejas. Cada G4 se dividirá en dos parejas.

Instrucciones para la realización del tutorial

Durante la realización del tutorial:

1. Debe ejecutar todas las operaciones que se le indican mediante el predicado "**Realice las siguientes operaciones"**.

2. El texto en cursiva contiene explicaciones relativas a elementos del lenguaje VHDL o reglas que debe aplicar en la realización de código. Léalas con atención y, si tiene cualquier duda o curiosidad sobre ellas, no dude en preguntar a su profesor.

3. Comunique a su profesor cualquier problema o incidencia con que se encuentre durante la realización del tutorial.

4. Cuando termine de realizar el tutorial comuníqueselo a su profesor.

PARTE II. Complejidad de los sistemas digitales

La complejidad es una característica que permite calificar la cantidad de elementos (módulos y canales de conexión) que forman la estructura de un sistema. Hay sistemas que no tienen estructura, en estos sistemas su funcionalidad no puede ser descrita mediante un conjunto de *subsistemas* (o *módulos* o *bloques*) independientes conectados entre sí; a este tipo de sistemas los denominamos *simples*.

Los *sistemas complejos* pueden ser descritos estructuralmente, como un conjunto de bloques conectados. El funcionamiento de un sistema complejo se deriva del de cada uno de los subsistemas que lo forman y del modo en que estos subsistemas interactúan. El grado de complejidad de un sistema depende del número de módulos que contenga su estructura, de la propia complejidad de dichos módulos y, también, del número de conexiones existentes entre ellos.

Pero la complejidad de un sistema es un concepto relativo, depende del lenguaje o método que se emplee al describirlo. Refiriéndonos ya al *mundo* de los sistemas digitales, podemos considerar la descripción de los sistemas combinacionales, que puede realizarse, por ejemplo, utilizando:

- Tablas de verdad

  Hay sistemas combinacionales cuyo funcionamiento puede describirse en una tabla de verdad completa o resumida (cualquier sistema con un número reducido de entradas) y los consideramos *sistemas combinacionales simples*, pero hay otros en los que resulta inviable abordar la realización de su tabla de verdad, como los subsistemas aritméticos, a los que consideramos *sistemas combinacionales complejos* y deben ser descritos como un conjunto de subsistemas simples -descritos, estos sí, mediante tablas de verdad- conectados entre sí.

- Circuitos lógicos

  Si extraemos las ecuaciones derivadas de la tabla de verdad de un sistema combinacional simple y realizamos con puertas un circuito lógico que represente las ecuaciones, dicho circuito tendrá estructura y, por tanto, a nivel de descripción con puertas es un circuito complejo. A nivel de descripción con puertas sólo podríamos considerar simples a los circuitos realizados con una sola puerta.

- Transistores

  Un inversor es, aisladamente, un circuito lógico simple, pero su descripción mediante un esquema que represente su realización con componentes electrónicos es compleja, ya que consiste en un conjunto de elementos (transistores, diodos, etcétera) conectados entre sí.

En resumen, un mismo sistema puede tener diferentes calificaciones en cuanto a su complejidad en función de las características del método empleado para representarlo. Los métodos descripción pueden calificarse en base al grado en que detallan o abstraen los detalles de la estructura del sistema que se describe. En la ingeniería de sistemas digitales se reconocen tres *niveles de abstracción* en las descripciones del modelo de funcionamiento de los sistemas:

- Nivel de transistor. Es el nivel de abstracción más bajo pues describe los sistemas en base a su estructura electrónica (a nivel de dispositivo). A este nivel todos los sistemas digitales son

complejos. Se emplea en la ingeniería de diseño y caracterización de las células lógicas que se integran en los chips y que se utilizan como módulos para la realización de circuitos lógicos.

- Nivel lógico. A este nivel los sistemas digitales se describen mediante ecuaciones o circuitos lógicos. En la actualidad su uso principal es la representación de resultados de operaciones del ciclo de diseño realizadas por herramientas de CAD –el propósito de utilizar este nivel de abstracción es informar de los resultados a un nivel detallado, pero susceptible de ser consultado o analizado con relativa facilidad por parte del diseñador.

- Nivel funcional. Con este nivel de abstracción se hace referencia a descripciones que, utilizando diversos lenguajes y métodos de representación de la funcionalidad (como tablas de verdad, cronogramas, lenguaje natural o HDLs), definen el funcionamiento de los sistemas con un grado de abstracción alto o muy alto. Los HDLs son el medio más empleado en la actualidad para la descripción funcional de sistemas digitales.

Complejidad de los modelos HDL

En VHDL pueden realizarse descripciones de funcionamiento a nivel lógico y funcional. Es un lenguaje que no permite realizar descripciones a nivel de transistor –con otros HDLs sí es posible realizar este tipo de descripciones. A nivel funcional pueden realizarse modelos de mayor o menor nivel de abstracción, depende del propósito con el que se realice el modelo. El nivel de abstracción de los modelos HDL sintetizables automáticamente es —dentro de los que se consideran como niveles de abstracción funcional— un nivel relativamente bajo; se conoce, en inglés, como *Register Transfer Level* (Nivel de Transferencias entre Registros) —de ahí viene el acrónimo que se suele utilizar como nombre de las arquitecturas de los modelos sintetizables, RTL.

En el nivel de descripción RTL —que es el que hemos empleado hasta aquí, y seguiremos empleando, en este curso— estamos considerando que un sistema digital es simple si su funcionamiento puede expresarse con un solo proceso. Por ejemplo, las técnicas de modelado de sistemas secuenciales que conoce hasta ahora sólo permiten abordar la descripción de sistemas cuya arquitectura se corresponda con los casos mostrados en la figura 1. Son los sistemas que hemos venido denominando sistemas secuenciales síncronos simples: flip-flops, registros y contadores, sistemas cuya característica común es que la salida es la memoria de estado.

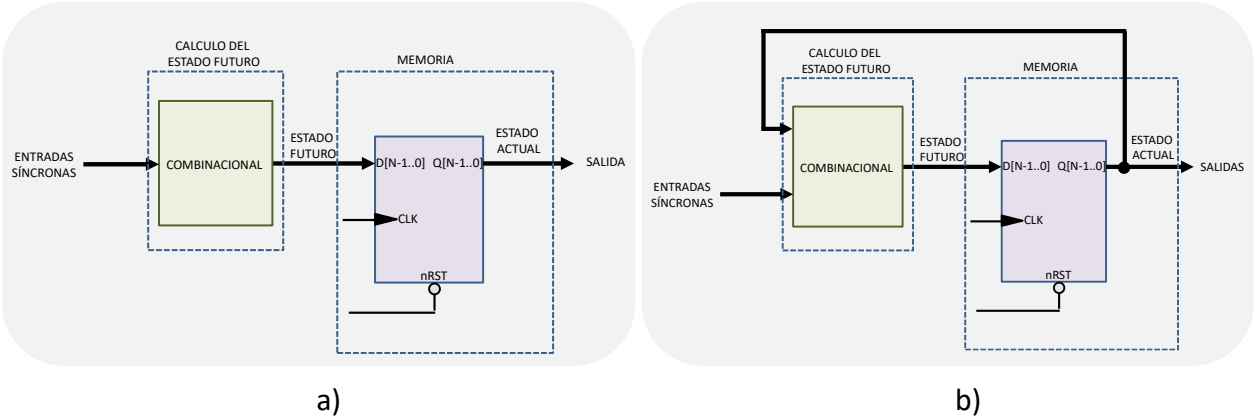


Figura 1.- Arquitecturas de Moore cuya salida es la memoria de estado

Modelado de sistemas secuenciales síncronos complejos

Pero esta técnica de modelado resulta insuficiente, por ejemplo, para describir sistemas en los que la salida se genera combinacionalmente a partir de los bits de la memoria de estado (sistemas con arquitectura de Moore) o a partir de la memoria de estado y las entradas síncronas (sistemas con arquitectura de Mealy). En la arquitectura de estos sistemas se distinguen los tres bloques que definen la estructura general de un autómata de Mealy o Moore (figura 2):

- la memoria de estado,
- la lógica de cálculo de transiciones y
- la lógica de cálculo de salidas.

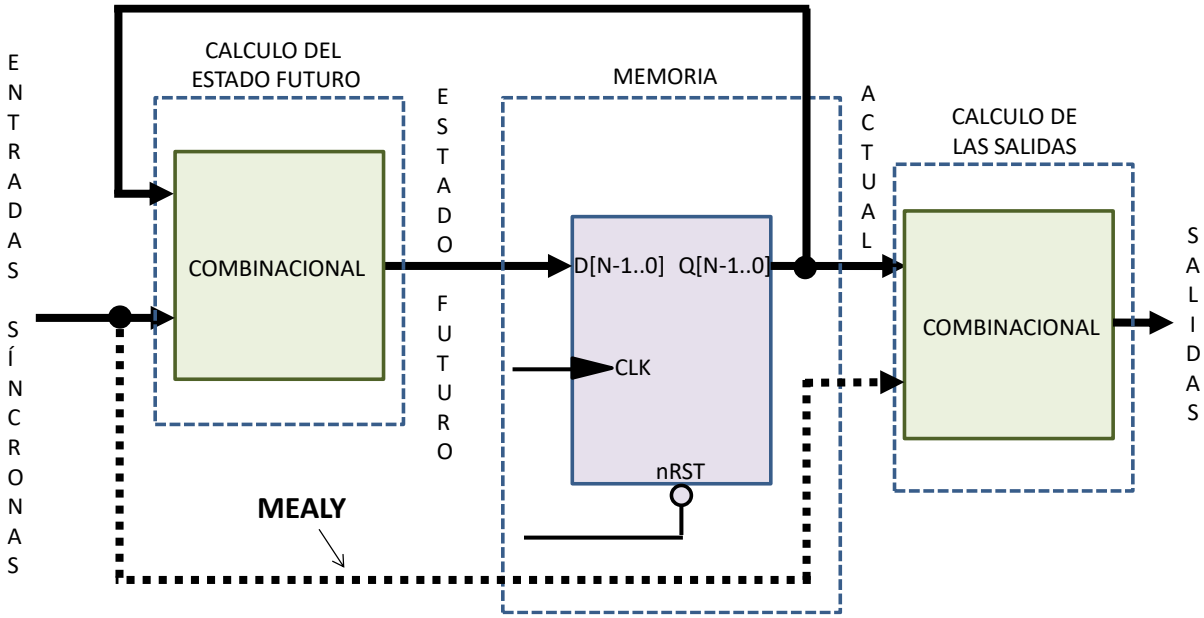


Figura 2.- Arquitectura general de un sistema secuencial síncrono

A nivel RTL, un sistema secuencial síncrono con una arquitectura como la de la figura 2 es considerado un sistema complejo, porque su modelado tiene que realizarse, como mínimo, con dos procesos:

- La memoria de estado y la lógica combinacional de cálculo de transiciones pueden modelarse utilizando un proceso idéntico a los que utilizan para modelar sistemas secuenciales simples.

- Para completar el modelo de la arquitectura hay que utilizar otro proceso más para modelar la lógica de cálculo de las salidas.

- Conectando ambos procesos se puede construir cualquier sistema con una arquitectura como la de la figura 2.

Esta técnica de modelado con dos procesos es aplicable a cualquier sistema secuencial síncrono (con arquitectura de Moore o Mealy). Su descripción y aplicación va a ocupar el siguiente apartado de este tutorial. Para ilustrarlo se utilizará como ejemplo la realización del modelo de un divisor de frecuencia.

PARTE III. Modelado de un divisor de frecuencia

Un divisor de frecuencia es un circuito secuencial con una salida cuya frecuencia es un factor de la frecuencia de la señal de reloj (figura 3a). Un divisor de frecuencia que divide la frecuencia de reloj por N es, simplemente, un contador de módulo N en el que la salida se obtiene detectando el paso del contador por uno de sus estados de cuenta (en la figura 3b se muestra el cronograma de un divisor por cuatro en el que se detecta el paso de la cuenta de un contador módulo cuatro por el estado de cuenta tres). Normalmente la salida se genera detectando el último estado y, por tanto, la salida del divisor se corresponde con la salida de fin de cuenta (FDC) del contador. En resumen, un divisor de frecuencia que divide la frecuencia de reloj por N resulta ser un contador módulo N con salida de fin de cuenta – que, por otra parte, es la única salida del circuito (figura 3c).

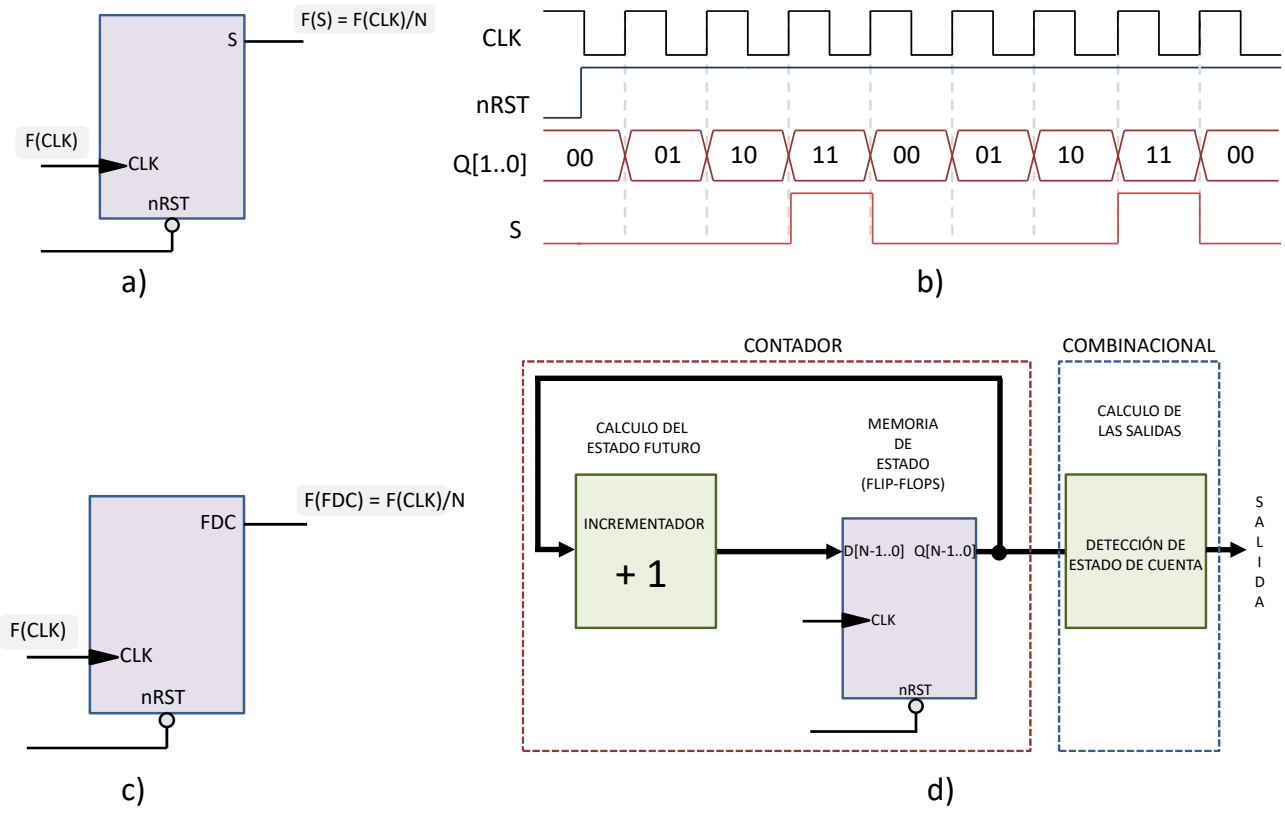


Figura 3.- Divisor de frecuencia

La arquitectura de un divisor de frecuencia como el que se ha descrito se corresponde con la de un autómata de Moore (figura 3d), en el que la lógica de cálculo de transiciones y la memoria de estado forman la estructura del contador módulo N y la lógica de cálculo de las salidas es un circuito combinacional que detecta el paso del contador por el último estado de cuenta.

Vamos a abordar, en concreto, la realización del modelo de un divisor de frecuencia con un factor de división 4. En primer lugar se va a proceder a modelar su interfaz.

Realice las siguientes operaciones:

1. Descargue de Moodle los ficheros correspondientes a esta actividad.

2. Arranque ModelSim y cree un proyecto, *Proy_ATGP4*, en el directorio de la actividad.

3. Cree un nuevo fichero VHDL, denomínelo *div_freq_4.vhd* y añádalo al proyecto.

   En la figura 4 se muestra el código correspondiente a la Declaración de Entidad que modela la interfaz de un divisor de frecuencia de módulo 4.

```
1   -- Modelo de un divisor de frecuencia por 4
2   -- realizado para la actividad AIP11
3   -- del  bloque I de Diseño digital I
4   -- (9-5-2011) versión 1.0
5
6   -- Cabecera: cláusulas de visibilidad (incluye unsigned)
7   library ieee;
8   use ieee.std_logic_1164.all;
9   use ieee.std_logic_unsigned.all;
10
11  -- Entidad: unidad primaria VHDL que modela la interfaz
12  entity div_freq_4 is
13  port(
14       clk:           in std_logic;
15       nRST:          in std_logic;
16       freq_div_4:    buffer std_logic
17       );
18  end entity;
19
```

Figura 4.- Declaración de Entidad del modelo del divisor de frecuencia

   En este código puede observarse que se añade la cláusula de visibilidad sobre el paquete *unsigned*, para poder utilizar operaciones aritméticas en el modelado del funcionamiento, y que la única salida del sistema es *freq_div_4*, una señal que debe tener una frecuencia igual a la del reloj dividida por cuatro.

4. Edite el fichero *div_freq_4.vhd* para añadir el código contenido entre las líneas 1 y 18 de la figura 4.

<u>Realización del modelo de funcionamiento de un sistema secuencial síncrono complejo</u>

La construcción del modelo de la estructura de un sistema secuencial síncrono complejo requiere de la realización de dos procesos: uno, similar al utilizado para describir sistemas secuenciales simples, debe modelar la memoria de estado y la lógica de cálculo de transiciones, el otro debe ser un proceso que siga las reglas de modelado de sistemas combinacionales, pues describe el funcionamiento de la lógica combinacional de salida.

La arquitectura del divisor de frecuencia por 4 está compuesta por dos subsistemas simples: un contador de módulo 4 y un combinacional que activa (pone a 1) su salida cuando su entrada es 3. El contador se describirá utilizando un proceso para el modelado de un sistema secuencial síncrono simple y el combinacional con un proceso para modelar combinacionales simples.
El contador de módulo 4 podría modelarse con el siguiente proceso:

```
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= "00";

  elsif clk'event and clk = '1' then
    Q <= Q + 1;

  end if;
end process;
```

Y el bloque combinacional podría modelarse con este otro:

```
process(Q)
begin
  if Q = 3 then
    freq_div_4 <= '1';

  else
    freq_div_4 <= '0';

  end if;
end process;
```

Dentro de la estructura de un sistema complejo, la conexión entre la salida de un módulo y la entrada de otro se realiza por medio de un nodo lógico (un nodo en el que puede observarse un nivel lógico). El nivel lógico de un nodo lo determina (lo *escribe*) la salida del módulo al que está conectado. La entrada del otro módulo detecta (puede *leer*) los cambios en el nivel del nodo lógico y reacciona cuando se producen. Mediante este mecanismo la interacción entre ambos módulos da lugar al funcionamiento característico del sistema complejo.

Los nodos lógicos que conectan los dos bloques en que hemos dividido la arquitectura de un sistema secuencial síncrono se conectan por medio de los nodos lógicos correspondientes a la memoria de estado del sistema. Para construir el modelo VHDL de un sistema secuencial síncrono complejo hay que disponer de objetos que puedan modelar nodos lógicos. Pues bien, en VHDL los nodos lógicos se modelan con señales. Hay que disponer de una señal con tantos bits como tenga la memoria de estado del sistema secuencial para modelar los nodos lógicos de conexión.

En el caso del divisor de frecuencia por 4, el contador y el circuito combinacional están conectados por dos nodos lógicos (codifican el estado de cuenta del contador). Estos nodos se modelan, en VHDL, declarando una señal de 2 bits en la Zona Declarativa del Cuerpo de Arquitectura. Por ejemplo, el siguiente código declara un bus, Q_i, compuesto por 2 nodos lógicos:

```
20
21  architecture rtl of div_freq_4 is
22    signal Q_i: std_logic_vector(1 downto 0);
23
24  begin
```

Figura 5

Realice la siguiente operación

1. Edite el fichero *div_freq_4.vhd* para añadir el código contenido entre las líneas 21 y 24 de la figura 5.

Cuando un nodo lógico está conectado a la salida de un módulo, dicho módulo es el que determina su nivel lógico; análogamente una señal VHDL (un nodo lógico), está conectada a la salida del proceso (módulo) que le asigna valor.

Por tanto, para conectar la señal Q_i a la salida del contador basta con que el proceso que lo modela sea el que asigne valor a Q_i:

```
21  architecture rtl of div_freq_4 is
22    signal Q_i: std_logic_vector(1 downto 0);
23
24  begin
25    -- Proceso que modela el funcionamiento del contador
26    process(clk, nRST)
27    begin
28      if nRST = '0' then
29        Q_i <= "00";
30
31      elsif clk'event and clk = '1' then
32        Q_i <= Q_i + 1;
33
34      end if;
35    end process;
36
```

Figura 6

Realice la siguiente operación

1. Edite el fichero *div_freq_4.vhd* para añadir el código contenido entre las líneas 25 y 35 de la figura 6.

Por otra parte, si un nodo lógico está conectado a la entrada de un módulo, dicho módulo reacciona a los cambios del nivel lógico del nodo, inmediatamente si es un módulo combinacional o en el siguiente flanco activo de reloj si es un módulo secuencial. Por tanto, para que una señal (un nodo lógico) esté conectada a la entrada de un proceso, el proceso debe reaccionar, según su naturaleza (combinacional o secuencial) a los cambios de nivel del nodo.

En definitiva, para conectar una señal a la entrada de un proceso basta con que sea tratada en el proceso tal y como se tratan las entradas en procesos que modelan sistemas simples

Para que Q_i sea una entrada del proceso que modela el módulo detector del divisor de frecuencia, dicho proceso debe ser sensible a Q_i y todos los valores de Q_i deben ser evaluados para determinar la salida del módulo (figura 7).

En la figura 7 se muestra el código completo del Cuerpo de Arquitectura del modelo del divisor de frecuencia.

```
21  architecture rtl of div_freq_4 is
22    signal Q_i: std_logic_vector(1 downto 0);
23
24  begin
25    -- Proceso que modela el funcionamiento del contador
26    process(clk, nRST)
27    begin
28      if nRST = '0' then
29        Q_i <= "00";
30
31      elsif clk'event and clk = '1' then
32        Q_i <= Q_i + 1;
33
34      end if;
35    end process;
36
37    --Proceso que detecta el estado de cuenta 3
38    process(Q_i)
39    begin
40      if Q_i = 3 then
41        freq_div_4 <= '1';
42
43      else
44        freq_div_4 <= '0';
45
46      end if;
47    end process;
48
49  end rtl;
50
```

Figura 7.-Cuerpo de arquitectura del divisor de frecuencia

Realice las siguientes operaciones:

1. Edite el fichero *div_freq_4.vhd* para añadir el código contenido entre las líneas 37 y 49 de la figura 7.

2. Salve y compile el modelo del divisor de frecuencia.

En resumen, para realizar el modelo VHDL de un sistema secuencial síncrono complejo hay que:

- Crear una Declaración de Entidad siguiendo las mismas reglas que en el caso de los sistemas digitales simples.
- Crear un Cuerpo de Arquitectura en el que se declare una señal, con tantos bits como tenga la memoria de estado del sistema, para modelar los nodos lógicos internos de conexión del sistema.
- Modelar (con un proceso idéntico al utilizado para modelar sistemas secuenciales simples) la memoria de estado y la lógica de transiciones y con otro proceso, combinacional, la lógica de cálculo de las salidas.
- Conectar ambos procesos con la señal declarada en el Cuerpo de Arquitectura, teniendo en cuenta que a dicha señal le debe asignar valor el proceso secuencial y debe ser considerada como una entrada por el proceso que modela la lógica combinacional de salida.

Una última observación: el orden en el que se escriban los procesos dentro del Cuerpo de Arquitectura es irrelevante, ya que su ejecución en una simulación es concurrente.

Ejecución del modelo en una simulación

Los procesos que forman parte del modelo de un sistema complejo se ejecutan en una simulación cuando se producen eventos en las señales a las que son sensibles. Por ejemplo, en el modelo del divisor de frecuencia, el proceso *secuencial* que modela el contador se ejecuta cuando la señal de reloj o la entrada de reset asíncrono cambian de valor, mientras que el proceso *combinacional* que detecta el estado de cuenta 3 se ejecuta en los eventos de la señal Q_i.

Lo que resulta novedoso en la ejecución de estos modelos es que hay procesos cuya ejecución no es provocada directamente por los estímulos de la simulación, sino por la actividad de señales manejadas por otros procesos. Es el caso, por ejemplo, del proceso *combinacional* del modelo del divisor: se ejecuta sólo cuando el proceso *secuencial* cambia el valor de Q_i. Este mecanismo es el que materializa la interacción entre procesos que representa la interacción entre módulos de la arquitectura modelada. Vamos a ilustrar esta dinámica de ejecución tomando como ejemplo la respuesta del modelo del divisor de frecuencia en una simulación como la representada en la figura 8.

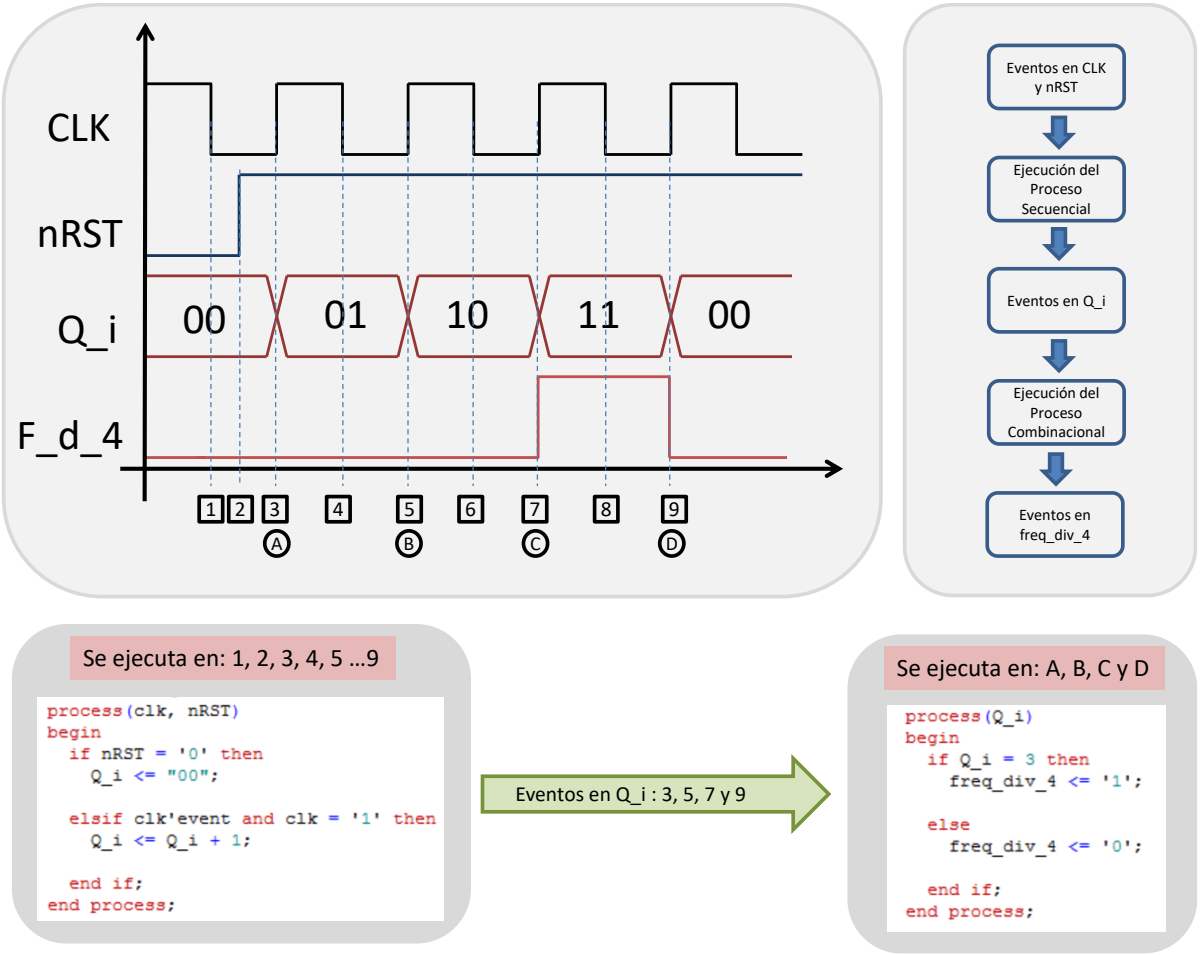


Figura 8.- Simulación de un modelo complejo

El cronograma muestra la respuesta del modelo del divisor a los estímulos generados en un *test-bench*. El proceso *secuencial* que modela el contador se ejecuta cuando hay eventos en CLK o nRST, en los nueve instantes marcados con los dígitos del 1 al 9. El resultado de la ejecución del proceso depende del evento: en los flancos negativos de reloj y en la desactivación del reset, la ejecución del proceso no da lugar a la proyección de asignaciones sobre Q_i, pero en los cuatro flancos positivos de reloj

(instantes 3, 5, 7 y 9), se ejecuta la sentencia Q_i <= Q_i + 1, que da lugar a que se proyecte un incremento del valor de Q_i. En estos cuatro instantes se repite la secuencia de sucesos que ilustra el diagrama de flujo situado a la derecha del cronograma en la figura 8:

- Hay un evento en CLK que se corresponde con un flanco de subida de reloj que da lugar a la ejecución de un ciclo de simulación.

- En la primera parte del ciclo de simulación se ejecuta el proceso secuencial (sensible a CLK), y dicha ejecución incluye la de la sentencia Q_i <= Q_i + 1, por lo que se proyecta una asignación de valor sobre Q_i.

- En la segunda parte del ciclo de simulación se actualiza Q_i a consecuencia de la asignación proyectada en la primera parte del ciclo de simulación. El cambio (incremento) en el valor de Q_i da lugar a la ocurrencia de un evento en esta señal. Como el proceso *combinacional* es sensible a Q_i se desencadena un ciclo delta de simulación, en cuya primera parte se ejecuta dicho proceso.

- La ejecución del proceso combinacional da lugar a la proyección de la asignación del valor '0' ó '1' sobre *freq_div_4*, dependiendo de si Q_i es distinto de 3 (lo que ocurre en los instantes marcados como A, B y D en el cronograma de la figura 8) o igual a 3 (esto sólo ocurre en el instante C).

- En la segunda parte del ciclo delta de simulación se actualiza *freq_div_4* con el valor proyectado; como no hay procesos sensibles a esta señal, no se ejecutan nuevos ciclos de simulación.

En definitiva, el mecanismo de ejecución de los procesos con listas de sensibilidad hace que los cambios de nivel lógico que produce un proceso en las señales a las que asigna valor desencadene la ejecución inmediata de procesos sensibles a dicha señal. Por ello, si cada proceso modela un módulo de una estructura conectada, el funcionamiento observado en la simulación es semejante al que tendría el hardware modelado, gracias al papel que desempeñan las señales VHDL, que resulta ser, efectivamente, equivalentes a los nodos lógicos de conexión de módulos hardware.

Realice las siguientes operaciones:

1. Añada al proyecto ModelSim el fichero *div_freq_4_tb.vhd* que encontrará en el directorio de la actividad.

   Este fichero contiene un *test-bench* para simular el modelo del divisor de frecuencia.

2. Compile el fichero y arranque el entorno de simulación.

Para poder analizar detalladamente el desarrollo de la simulación resulta conveniente poder observar los valores que va tomando la señal Q_i. El siguiente procedimiento describe como se añaden señales internas al visor de formas de onda.

Realice las siguientes operaciones:

Cuando se arranca el entorno de simulación, la ventana *Objects* muestra los objetos declarados en el *test-bench*: las señales que se conectan al modelo emplazado para actuar como estímulos o para recibir la respuesta de las salidas.

1. Añada las señales del *test-bench* al visor de formas de onda.

2. La señal Q_i está declarada en el modelo del divisor. Para que aparezca en la ventana *Objects* y pueda añadirla al visor, seleccione en la ventana *Sim* la línea correspondiente al *dut* (figura 9).

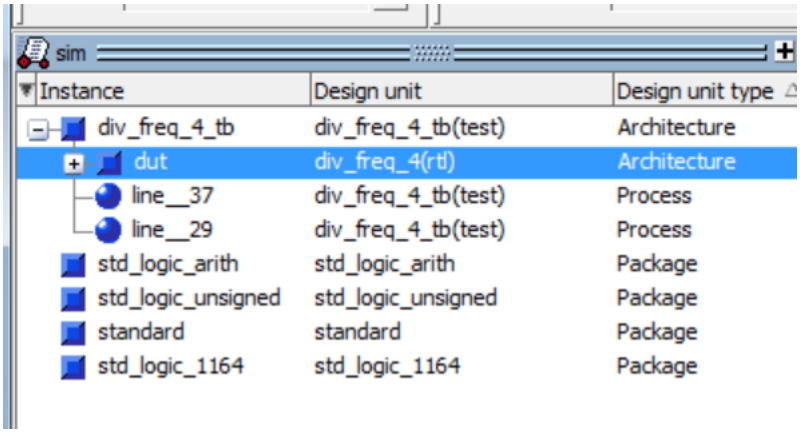


Figura 9.- Cambio del contexto de selección

La información que aparece en las ventanas del entorno de simulación depende del contexto que se encuentre seleccionado. La última acción ha cambiado el contexto del *test-bench* (que es con el que arranca por defecto) al modelo. Por eso la ventana *Object* muestra ahora los objetos declarados en el modelo: sus puertos y la señal Q_i.

3. Seleccione Q_i y añádala al visor de formas de onda.

4. Ejecute una simulación con una duración de 2 microsegundos y verifique que el resultado es el esperado.

5. Finalice la simulación.

PARTE IV. Modelado de sistemas complejos

La complejidad del modelo del divisor de frecuencia es mínima, ya que su arquitectura consta de dos módulos que interaccionan por medio de dos nodos lógicos (los dos bits de cuenta). Pero el procedimiento que se ha utilizado para construirlo es generalizable al modelado de la arquitectura de cualquier sistema, sea cual sea su complejidad.

En este apartado se explicará el modelado de un sistema más complejo, compuesto por cuatro bloques funcionales interconectados. La descripción VHDL de este modelo se encuentra en la carpeta de la actividad.

Realice las siguientes operaciones:

1. Añada al proyecto ModelSim el fichero *oper_reg.vhd* que encontrará en la carpeta de la actividad.

2. Compile el fichero y visualice su contenido.

Como puede ver, la declaración de entidad del modelo de este sistema se realiza como el de cualquier sistema digital simple. El siguiente código modela la interfaz del sistema:

```
-- Cabecera: cláusulas de visibilidad (incluye unsigned)
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

-- Entidad: unidad primaria VHDL que modela la interfaz
entity oper_reg is
port(
     clk:    in std_logic;
     nRST:   in std_logic;
     ena_A:  in std_logic;
     ena_B:  in std_logic;
     sel:    in std_logic;
     A:      in std_logic_vector(3 downto 0);
     B:      in std_logic_vector(3 downto 0);
     D_out:  buffer std_logic_vector(3 downto 0);
     );
end entity;
```

En el Cuerpo de Arquitectura del sistema hay que declarar las señales que modelan los nodos lógicos internos de conexión. Este circuito requiere dos buses internos, REG_A y REG_B, y un nodo, A_gr_B:

```
architecture rtl of oper_reg is
  signal reg_A: std_logic_vector(3 downto 0);
  signal reg_B: std_logic_vector(3 downto 0);
  signal A_gr_B: std_logic;
```

Dentro del cuerpo de arquitectura se han descrito cuatro procesos, el primero de ellos es:

```
process(clk, nRST)
begin
  if nRST = '0' then
    reg_A <= "0000";

  elsif clk'event and clk = '1' then
    if ena_A = '1' then
     reg_A <= A;

    end if;
  end if;
end process;
```

Realice las siguientes operaciones:

1. Identifique el tipo de circuito modelado por este proceso e indíquelo mediante un comentario en el código VHDL del sistema.

Observe que la conexión del nodo reg_A a la salida del circuito se materializa por el hecho de que el proceso que modela ese circuito asigna valor a reg_A.

El siguiente proceso modela un circuito con 3 entradas. Dos de ellas están manejadas por sendas entradas del sistema, mientras que la tecera está conectada al nodo interno A_gr_B:

```
process(clk, nRST)
begin
  if nRST = '0' then
    reg_B <= "0000";

  elsif clk'event and clk = '1' then
    if ena_B = '1' and A_gr_B = '1' then
     reg_B <= B;

    end if;
  end if;
end process;
```

Realice las siguientes operaciones:

1. Identifique el tipo de circuito modelado por este proceso e indíquelo mediante un comentario en el código VHDL del sistema.

La conexión del nodo A_gr_B a una de las entradas de este circuito se modela haciendo que el nodo reciba el tratamiento correspondiente a una entrada síncrona en el proceso que modela el circuito. El nodo reg_B está conectado a la salida del circuito porque el proceso le asigna valor.

El siguiente proceso tiene como entradas la salida del circuito modelado en el primer proceso y la entrada B del sistema; su salida está conectada al nodo A_gr_B. Por tanto, el proceso que modela al circuito debe ser sensible a reg_A y B, y los valores de ambos deben evaluarse para asignar valor a A_gr_B:

```
process(reg_A, B)
begin
  if reg_A > B then
    A_gr_B <= '1';

  else
    A_gr_B <= '0';
  end if;
end process;
```

Realice las siguientes operaciones:

Identifique el tipo de circuito modelado por este proceso e indíquelo mediante un comentario en el código VHDL del sistema.
El último proceso tiene como entradas los dos buses internos reg_A y reg_B, y la entrada del sistema SEL; su salida es D_out:

```
process(reg_A, reg_B, sel)
begin
  if sel = '0' then
    D_out <= reg_A;

  else
    D_out <= reg_B;
  end if;
end process;
```

El proceso es sensible a los dos buses internos y a la entrada sel (a todas las entradas del circuito) y asigna valor a D_out (la salida del circuito).

Realice las siguientes operaciones:

1. Identifique el tipo de circuito modelado por este proceso e indíquelo mediante un comentario en el código VHDL del sistema.

Realice las siguientes operaciones:

1. Dibuje un diagrama del sistema completo en el que aparezcan los cuatro circuitos que ha identificado, correctamente interconectados.

2. Complete el comentario en la cabecera del código VHDL que describe el sistema completo resumiendo su funcionamiento.

En resumen, las reglas que hay que seguir para realizar el modelo de un sistema complejo son:

- Crear una Declaración de Entidad siguiendo las mismas reglas que en el modelado de sistemas digitales simples.

- Crear un Cuerpo de Arquitectura en el que se declaren tantas señales como nodos internos de conexión haya en la estructura del sistema.

- Modelar cada subsistema de la arquitectura del sistema digital complejo con un proceso, modelando la conexión de entradas y salida y nodos internos que se defina en la arquitectura. Para ello debe tenerse en cuenta que:

    a. Las señales a las que asigna valor el proceso son nodos conectados a salidas del subsistema.

    b. Las señales que reciben el tratamiento propio de las entradas en el modelado de sistemas digitales simples son nodos conectados a las entradas del subsistema.

Realice las siguientes operaciones:

2. Añada al proyecto ModelSim el fichero *oper_reg_tb.vhd* que encontrará en el directorio de la actividad.

   Este fichero contiene el *test-bench* del sistema completo.

3. Compile el test y realice una simulación para comprobar el funcionamiento del modelo.